SLL

Code :

```
# Node class
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


# Singly Linked List
class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    # Traverse
    def traverse(self):
        if self.head is None:
            print("List is empty")
            return
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

    # Insert at end
    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            self.tail = new_node
        else:
```

```
        self.tail.next = new_node
        self.tail = new_node

# Insert at beginning
def insert_at_beginning(self, data):
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
        self.tail = new_node
    else:
        new_node.next = self.head
        self.head = new_node

# Insert at middle (specific position)
def insert_at_position(self, data, pos):
    new_node = Node(data)

    # Insert at beginning
    if pos == 1:
        self.insert_at_beginning(data)
        return

    current = self.head
    count = 1

    # Traverse until (pos-1)th node
    while current is not None and count < pos - 1:
        current = current.next
        count += 1

    # If position is invalid (greater than list length+1)
    if current is None:
        print("Position out of range")
        return
```

```
        # Insert node in middle or at end
        new_node.next = current.next
        current.next = new_node

        # If inserted at last, update tail
        if new_node.next is None:
            self.tail = new_node


# ------------------------
# Testing
# ------------------------
ll = SinglyLinkedList()

# Insert at end
ll.insert_at_end(10)
ll.insert_at_end(20)
ll.insert_at_end(40)

# Insert at beginning
ll.insert_at_beginning(5)

# Insert at position
ll.insert_at_position(30, 4)  # Insert 30 at 4th position
ll.insert_at_position(1, 1)   # Insert 1 at beginning
ll.insert_at_position(50, 7)  # Insert 50 at end

# Traverse
ll.traverse()

print("Head:", ll.head.data)
print("Tail:", ll.tail.data)
```